Tree

沈威宇

November 13, 2026

1 Tree

I Tree

i Tree

A tree is an abstract data type of a set of nodes. A relationship between two nodes i, j is defined with

- i is called a child of j and
- j is called a parent of i; and
- a node can have at most one parent.

If not empty, a tree is recursively defined as a distinguished node r called root and zero or more nonempty trees called subtrees, where the root of each substree is a child of r.

ii Terminology

- Siblings: Nodes sharing the same parent are called siblings. Children of a node are defined with a left to right order.
- Leaf: A node with no children is called a leaf, external node, or exterior node.
- Internal node: A node that is not a leaf is called an internal node or interior node.
- Depth: The number of edges from the root to a node is called the depth of the node, while the root has depth 0.
- Height: The height of a leaf is defined as 0. The height of an internal node is defined as the maximum height of its children plus 1. The maximum depth of the nodes, i.e., the height of its root, is called the height or depth of the tree and denoted h. The height or depth of an empty tree is defined as -1. Some define heights of nodes and trees as the definition here plus 1, which is not used here.
- Ancestors: The ancestors of a node is recursively defined as itself for root and itself and the ancestors of its parent for non-root nodes.
- Descendants: The descendants of a node is recursively defined as itself for leaves and itself and the descendants of its children for internal nodes.
- Level: Nodes of the same depth are called a level. Levels are ordered by depth. The left-to-right order of nodes in a level is defined recursively to, if they are siblings, follow that of siblings; otherwise follow that of their parents.
- Subtree: A tree consisting of all descendants of a node is called a subtree. The left-to-right order of subtrees whose roots are of the same depth are defined as

that of their roots. A subtree whose root is a child of a node is called a subtree of the node.

- The umber of nodes in a tree is denoted n.

iii Tree representation

A representation of tree where each pair of parent and child is connected with a edge.

iv Left child, right sibling representation

A representation of tree where each internal node and its leftmost child is connected with a edge, and each sibling that is not the leftmost one is called to the sibling at its left.

v Skewed

A tree whose every level contains only one node is called skewed.

II n-ary tree

i n-ary tree

A tree in which no node can have more than n children is called an n-ary tree. A child is indexed into one and only one integer in $[1, n]$ even if its parent has less than n children.

ii Full, proper, or plane

An n-ary tree where each internal node has exactly n children is called full, proper, or plane.

iii Filled

A level of depth $k \in \mathbb{N}$ of an n-ary tree is called filled iff each node in the level of depth $k - 1$ has exactly n children. The level of depth 0 is called filled iff the tree is nonempty.

iv Complete

An n-ary tree where every level, except possibly the last, is filled and whose nodes in the last level are as far left as possible, is called complete.

A complete n-ary tree is indexed from top to bottom and then from left to right, with the root as node 1.

v Perfect

A complete n-ary tree where all leaves are of the same depth is called perfect.

A perfect n-ary tree with height k has $\sum_{j=0}^{k} n^j$ nodes.

III Binary tree

i Binary tree

In a binary tree, the two possible children of a node are called left child and right child. A child is either left or right but not both even if it has no sibling. The left and right subtrees of a node x are denoted left(x) and right(x) respectively.

ii Complete binary tree

For a complete binary tree with n nodes:

- Parent of node i is node $\left\lfloor \frac{i}{2} \right\rfloor$ for all integers $i > 1$.
- The left child of node i is $2i + 1$ for all integers $i \leq \frac{n-1}{2}$.
- The right child of node i is $2i$ for all integers $i \leq \frac{n}{2}$.

iii Perfect binary tree

A perfect binary tree with height k has $2^{k+1} - 1$ nodes.

iv Balanced tree

A type of binary tree with height $O(\log n)$ where n is the number of nodes is called balanced.

v Balance factor (bf)

The height of the left subtree minus the height of the right subtree of a node in a binary tree is called the balance factor of the node.

vi Left-skewed and right-skewed

A binary tree whose every internal node has left child and does not have right child is called left-skewed.

A binary tree whose every internal node has right child and does not have left child is called right-skewed.

IV Tree traversal

They are defined recursively. Suppose the root is r.

i Preorder traversal

First visit r, and then visit the subtrees of r from left to right.

ii Postorder traversal

Visit the subtrees of r from left to right, and finally visit r.

iii Inorder traversal (binary tree)

First visit the left subtree of r, and then visit r, and then visit the right subtree of r.

iv Full binary tree with distinct values can be uniquely determined by its preorder and postorder traversals

A full binary tree with distinct values can be uniquely determined by its preorder and postorder traversals.

Proof. The following C code provides an algorithm to reconstruct the tree recursively:

```c
#include <stdlib.h>

typedef struct Node {
  int data;
  struct Node *left;
  struct Node *right;
} Node;

Node *createNode(int data) {
  Node *n = (Node *)malloc(sizeof(Node));
  n->data = data;
  n->left = NULL;
  n->right = NULL;
  return n;
}

/*
 * Reconstruct a full binary tree from:
 *   P - preorder array (size num)
 *   Q - postorder array (size num)
 *   distinct values assumed
 */
Node *construct(int *P, int *Q, int num) {
  if (num <= 0)
    return NULL;
  if (num == 1)
    return createNode(P[0]);
  Node *root = createNode(P[0]);
  int leftRootVal = P[1];
  int k = 0;
  while (Q[k] != leftRootVal)
```

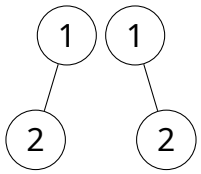
```
    k++;
  int leftSize = k + 1;
  int rightSize = num - 1 - leftSize;
  root->left = construct(P + 1, Q, leftSize);
  root->right = construct(P + 1 + leftSize, Q + leftSize, rightSize);
  return root;
}
```

□

v Binary tree with distinct values cannot be uniquely determined by its preorder and postorder traversals

The statement that a binary tree with distinct values can be uniquely determined by its preorder and postorder traversals is false.

Proof. No. A counterexample is the following two binary trees



Both trees have preorder traversal 1, 2, and postorder traversal 2, 1, but they are different. □

vi Binary tree with distinct values can be uniquely determined by its preorder and inorder traversals

A binary tree with distinct values can be uniquely determined by its preorder and inorder traversals.

Proof. The following C code provides an algorithm to reconstruct the tree recursively:

```c
#include <stdlib.h>

typedef struct Node {
  int data;
  struct Node *left;
  struct Node *right;
} Node;

Node *createNode(int data) {
  Node *n = (Node *)malloc(sizeof(Node));
  n->data = data;
  n->left = NULL;
  n->right = NULL;
  return n;
}

/*
 * Reconstruct a binary tree from:
```

```
 *   P - preorder array (size num)
 *   I - inorder array (size num)
 *   distinct values assumed
 */
Node *construct(int *P, int *I, int num) {
  if (num <= 0)
    return NULL;
  if (num == 1)
    return createNode(P[0]);
  int rootVal = P[0];
  Node *root = createNode(rootVal);
  int k = 0;
  while (I[k] != rootVal)
    k++;
  int leftSize = k;
  int rightSize = num - 1 - leftSize;
  root->left = construct(P + 1, I, leftSize);
  root->right = construct(P + 1 + leftSize, I + leftSize + 1, rightSize);
  return root;
}
```

□

vii Binary tree with distinct values can be uniquely determined by its postorder and inorder traversals

A binary tree with distinct values can be uniquely determined by its postorder and inorder traversals.

Proof. The following C code provides an algorithm to reconstruct the tree recursively:

```
#include <stdlib.h>

typedef struct Node {
  int data;
  struct Node *left;
  struct Node *right;
} Node;

Node *createNode(int data) {
  Node *n = (Node *)malloc(sizeof(Node));
  n->data = data;
  n->left = NULL;
  n->right = NULL;
  return n;
}

/*
 * Reconstruct a binary tree from:
 *   Q - postorder array (size num)
 *   I - inorder array (size num)
 *   distinct values assumed
 */
Node *construct(int *Q, int *I, int num) {
  if (num <= 0)
    return NULL;
  if (num == 1)
    return createNode(Q[num - 1]);
```

```
  int rootVal = Q[num - 1];
  Node *root = createNode(rootVal);
  int k = 0;
  while (I[k] != rootVal)
    k++;
  int leftSize = k;
  int rightSize = num - 1 - k;
  root->left = construct(I, Q, leftSize);
  root->right = construct(I + k + 1, Q + leftSize, rightSize);
  return root;
}
```

□

V Binary search tree (BST)

i Binary search tree (BST)

A binary search tree stores keys in the nodes of a binary tree so that searching, insertion and deletion can be done efficiently. For every node x, all the keys in its left subtree are smaller than x and all keys in its right subtree are larger than x. Thus, the values are distinct.

ii Searching

Suppose we are searching for x and the root is r.

- If $x = r$, then we are done.
- If $x < r$, then we search in the left subtree.
- If $x > r$, then we search in the right subtree.

The time complexity is $O(h)$.

iii Finding minimum and maximum

To find the node containing the smallest key, start at root and go to the left child as long as there is one, and the leaf we stop at is the smallest key. Similarly for find the largest key.

The time complexity is $O(h)$.

iv Insertion

Suppose we want to insert x. First search for it, if we found it, do noting (or update something); otherwise, if we stop at y,

- if $x > y$, insert x as the right child of y if $x > y$, and
- if $x < y$, insert x as the left child of y.

The time complexity is $O(h)$.

v Deletion

Suppose we want to delete x. First search for it, if we don't found it, do noting (or update something); otherwise,

- if it is a leaf, delete it,
- if it has one child y, adjust y as a child of the parent of x of the same position as x and delete x, and
- if it has two children, choose either of the policies:
  - inorder successor: replace the value of x with the value of the minimum element y in the right subtree of x, i.e. inorder successor, i.e., the leftmost descendant of the right child of x, which has no left child but may have right child, and
    - remove node y if it has no child, and
    - replace node y with its right child if it has one;
  - inorder predecessor: replace the value of x with the value of the maximum element y in the left subtree of x, i.e. inorder predecessor, i.e., the rightmost descendant of the left child of x, which has no right child but may have left child, and
    - remove node y if it has no child, and
    - replace node y with its left child if it has one;

The time complexity is $O(h)$.

vi (Simple) rotation

A (simple) rotation on an edge between a node x and its parent p, also called rooted at or on p, with p called root and x called pivot of the rotation, is a tree operation on BST and has the following two symmetric variants:

- Right rotation: This is done when x is a left child of p. We perform the following assignments:

$$\text{left}(p) = \text{right}(x)$$

$$\text{right}(x) = p$$

- Left rotation: This is done when x is a right child of p. We perform the following assignments:

$$\text{right}(p) = \text{left}(x)$$

$$\text{left}(x) = p$$

vii Uniqueness given preorder traversal

A BST is uniquely determined by its preorder traversal.

Proof. A BST with one node is obviously unique. The first element must be root, the second element to the last element smaller than root must be the left subtree, and the rest must be the right subtree. Recursively partition into the subtrees. We uniquely obtained the BST. □

viii Uniqueness given postorder traversal

A BST is uniquely determined by its postorder traversal.

Proof. A BST with one node is obviously unique. The last element must be root, the first element to the last element smaller than root must be the left subtree, and the rest must be the right subtree. Recursively partition into the subtrees. We uniquely obtained the BST. □

ix Binary tree is BST iff inorder traversal is in ascending order

A binary tree with distinct values is a binary search tree iff its inorder traversal is sorted in ascending order.

Proof. Only if: Prove by mathematical induction. The inorder traversal of a BST with one node is sorted in ascending order. Assume the inorder traversal of any BST with fewer than n nodes of distinct values is sorted in ascending order. Consider any BST with n nodes, the inorder traversal visits the left subtree, which is smaller than root and itself sorted in ascending order since it's a BST with $n - 1$ nodes, and then root, and then the right subtree, which is larger than root and itself sorted in ascending order since it's a BST with $n - 1$ nodes. Thus, we conclude that the inorder traversal of any BST with n nodes is sorted in ascending order. By mathematical induction, the statement holds.

If: Prove by mathematical induction. A binary tree with one node is a BST. Assume that any binary tree with fewer than n nodes of distinct values whose inorder traversal is sorted in ascending order is a BST. Consider any binary tree with n nodes of distinct values and root r whose inorder traversal is sorted in ascending order. The subsequence before r is a binary tree with fewer than n nodes of distinct values whose inorder traversal is sorted in ascending order whose values of nodes are smaller than r. The subsequence after r is a binary tree with fewer than n nodes of distinct values whose inorder traversal is sorted in ascending order whose values of nodes are larger than r. Since the left subtree is a BST and smaller than root and the right subtree is a BST and larger than root, we conclude that any binary tree with n nodes of distinct values whose

inorder traversal is sorted in ascending order is a BST. By mathematical induction, the statement holds. □

x Number of distinct BST shapes

The number T_n of distinct shapes of BSTs with n nodes is:

$$T_0 = 1$$

$$T_n = \sum_{i=0}^{n-1} T_i T_{n-1-i}, \quad n \in \mathbb{N}$$

$$T_n = \frac{1}{(n+1)} \binom{2n}{n}, \quad n \in \mathbb{N}_0$$

Proof. Obviously

$$T_0 = T_1 = 1$$

Let the values be

$$k_1 < k_2 < \ldots < k_n$$

Suppose we choose k_i as root, where $1 \leq i \leq n$. The left subtree contains $k_1, \ldots, k_{i-1}$, so it has $i-1$ nodes and T_{i-1} possible combinations. The right subtree contains $k_{i+1}, \ldots, k_n$, so it has $n-1$ nodes and T_{n-i} possible combinations.

Summing over all choices gives

$$T_n = \sum_{i=1}^{n} T_{i-1} T_{n-i} = \sum_{i=0}^{n-1} T_i T_{n-1-i}$$

Using generating function:

$$C(x) = \sum_{n=0}^{\infty} T_n x^n$$

$$C(x) = 1 + xC(x)^2$$

$$C(x) = \frac{1 \pm \sqrt{1-4x}}{2x}$$

$$C(0) = \mp \frac{1}{\sqrt{1-4x}} = T(0) = 1$$

$$C(x) = \frac{1 - \sqrt{1-4x}}{2x}$$

$$
\begin{aligned}
\sqrt{1-4x} &= \sum_{k=0}^{\infty} \binom{0.5}{k} (-4x)^k \\
&= 1 + \sum_{k=1}^{\infty} \frac{\prod_{i=0}^{k-1}(0.5-i)}{k!}(-4x)^k \\
&= 1 + \sum_{k=1}^{\infty} \frac{2^k \prod_{i=0}^{k-1}(2i-1)}{k!}x^k \\
&= 1 + \sum_{k=1}^{\infty} -\frac{2^k \prod_{i=1}^{k-1}(2i-1)}{k!}x^k \\
&= 1 + \sum_{k=1}^{\infty} -\frac{2^k \prod_{i=1}^{2k-2} i}{k! \prod_{i=1}^{k-1} 2i}x^k \\
&= 1 + \sum_{k=1}^{\infty} -\frac{2^k (2k-2)!}{k! 2^{k-1}(k-1)!}x^k \\
&= 1 + \sum_{k=1}^{\infty} -\frac{2(2k-2)!}{k!(k-1)!}x^k
\end{aligned}
$$

$$
\begin{aligned}
C(x) &= \frac{1-\sqrt{1-4x}}{2x} \\
&= \sum_{k=1}^{\infty} \frac{(2k-2)!}{k!(k-1)!}x^{k-1} \\
&= \sum_{n=0}^{\infty} \frac{(2n)!}{(n+1)!n!}x^n \\
&= \sum_{n=0}^{\infty} \frac{1}{(n+1)}\binom{2n}{n}x^n
\end{aligned}
$$

$$
T_n = \frac{1}{(n+1)}\binom{2n}{n}, \quad n \in \mathbb{N}_0
$$

□

VI AVL tree (Adelson-Velsky and Landis tree)

i AVL tree (Adelson-Velsky and Landis tree)

An AVL tree is a self-balancing binary search tree (BST) that keeps info of height, invented by Georgy Adelson-Velsky and Evgenii Landis. An AVL tree needs to comply with the property that for every node with two children in the tree, the height of its left and right subtrees differ by at most 1.

Some define heights of nodes and AVL trees as the definition used here plus 1, which is not used here.

ii Relation of height and numbers of nodes

Let the minimum number of nodes in an AVL tree of height h be N_h. Since the minimum numbers of subtrees height of the root of an AVL tree of height h are $h-1$ and $h-2$, we have:

$$N_{-1} = 0$$

$$N_0 = 1$$

$$N_h = N_{h-1} + N_{h-2} + 1, \quad h \in \mathbb{N}$$

$$N_h = \frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^{h+3} - \left(\frac{1-\sqrt{5}}{2}\right)^{h+3}\right) - 1, \quad h+1 \in \mathbb{N}_0$$

Proof. Use characteristic equation:

$$x^2 - x - 1 = 0$$

Let

$$\varphi = \frac{1+\sqrt{5}}{2}$$

Note that

$$\frac{3+\sqrt{5}}{2} = \varphi^2 = \varphi + 1$$

$$2 + \sqrt{5} = \varphi^3 = 2\varphi + 1$$

$$\frac{7+3\sqrt{5}}{2} = \varphi^4 = 3\varphi + 2$$

$$\frac{1-\sqrt{5}}{2} = -\frac{1}{\varphi} = 1 - \varphi$$

$$\frac{3-\sqrt{5}}{2} = \frac{1}{\varphi^2} = 1 - \frac{1}{\varphi} = 2 - \varphi$$

$$x = \frac{1 \pm \sqrt{5}}{2} = \varphi, -\frac{1}{\varphi}$$

$$N_h = a\varphi^h + b\left(-\frac{1}{\varphi}\right)^h + c$$

$$N_0 = a + b + c = 1$$

$$N_1 = a\varphi - \frac{b}{\varphi} + c = 0 + 1 + 1 = 2$$

$$N_2 = a\varphi^2 + \frac{b}{\varphi^2} + c = 1 + 2 + 1 = 4$$

$$N_2 = a(\varphi + 1) - b\left(\frac{1}{\varphi} - 1\right) + c = 4$$

$$N_2 - N_1 = a + b = 4 - 2 = 2$$

$$c = N_0 - (a + b) = 1 - 2 = -1$$

$$N_1 - c = a\varphi - \frac{2-a}{\varphi} = 2 + 1 = 3$$

$$a\left(\varphi^2 + 1\right) = 3\varphi + 2$$

$$a = \frac{3\varphi + 2}{\varphi + 2} = \frac{\varphi^4}{\varphi + 2}$$

$$b = 2 - a = \frac{2-\varphi}{\varphi+2} = \frac{1}{\varphi^2(\varphi+2)}$$

$$N_h = \frac{1}{\varphi+2}\left(\varphi^{h+4} + \left(-\frac{1}{\varphi}\right)^{h+2}\right) - 1$$

$$\frac{1}{\varphi+2} = \frac{2}{5+\sqrt{5}} = \frac{1}{\sqrt{5}\varphi}$$

$$N_h = \frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^{h+3} - \left(\frac{1-\sqrt{5}}{2}\right)^{h+3}\right) - 1$$

□

So

$$N_h = \Theta(\left(\frac{1+\sqrt{5}}{2}\right)^h)$$

$$h = \log_{\frac{1+\sqrt{5}}{2}} N_h + O(1) \approx 1.44 \log_2 N_h + O(1)$$

Let the maximum number of nodes in an AVL tree of height h be M_h. Since the maximum numbers of subtrees height of the root of an AVL tree of height h are $h-1$, we have:

$$M_{-1} = 0$$

$$M_h = 2M_{h-1} + 1, \quad h \in \mathbb{N}_0$$

$$M_h = 2^{h+1} - 1, \quad h + 1 \in \mathbb{N}_0$$

So

$$M_h \in \Theta(\log_2 M_h)$$

$$h = \log_2 M_h + O(1)$$

Let the number of nodes be n.

$$N_h \le n \le M_h$$

$$\log_2 n + O(1) \le h \le \log_{\frac{1+\sqrt{5}}{2}} n + O(1) \approx 1.44 \log_2 N_h + O(1)$$

Thus

$$h \in \Theta(\log n)$$

iii Searching and finding minimum/maximum

Searching and finding minimum/maximum in AVL tree is the same as in BST.

The worst-case and amortized time complexities are $O(h) = O(\log n)$.

iv Rotation

When insertion/deletion, the AVL property may be violated, called unbalanced. Since an insertion/deletion involves adding/deleting a single node, this can only increase/decrease the height of some subtree by 1. Thus, if the AVL property is violated at a node x, it means that the heights of left and right subtrees of it differ by exactly 2. We need to transform the tree to restore the AVL property, called rebalancing, using a single/simple rotation or a double rotation, each has two symmetric variants:

- LL case / right single rotation: Let the left child of x be y. The LL case is where the left subtree of y has height $h+1$, while the right subtree of y and the right subtree of x have height h. We perform a right rotation on the edge between y and x, i.e.,

$$\text{left}(x) = \text{right}(y)$$
$$\text{right}(y) = x$$

- RR case / left single rotation: Let the right child of x be y. The RR case is where the right tree of y has height $h+1$, while the left subtree of y and the left subtree of x have height h. We perform a left rotation on the edge between y and x, i.e.,

$$\text{right}(x) = \text{left}(y)$$
$$\text{left}(y) = x$$

- LR case / left-then-right double rotation: Let the left child of x be y and the right child of y be z. The LR case is where the right subtree of y has height $h+1$, while the left subtree of y and the right subtree of x have height h. We perform a left rotation on the edge between z and y and then a right rotation on the edge between z, now at the original position of y, and x, i.e.,

$$\text{right}(y) = \text{left}(z)$$
$$\text{left}(z) = y$$
$$\text{left}(x) = \text{right}(z)$$
$$\text{right}(z) = x$$

- RL case / right-then-left double rotation: Let the right child of x be y and the left child of y be z. The RL case is where the left subtree of y has height $h+1$, while the right subtree of y and the left subtree of x have height h. We perform a right rotation on the edge between z and y and then a left rotation on the edge between z, now at the original position of y, and x, i.e.,

$$\text{left}(y) = \text{right}(z)$$
$$\text{right}(z) = y$$
$$\text{right}(x) = \text{left}(z)$$
$$\text{left}(z) = x$$

v Insertion

Insertion in AVL tree involves first doing insertion as with a normal BST, and then tracing the path from the new leaf to the root: For each node x encountered, check if heights of its left and right subtrees differ by at most 1,

- if yes, proceed to the parent of x;
- if not, we say the AVL property is violated at x and perform rotations to restore it, and then break the loop since no second violation may exist.

Proof. Let x be the first node encountered when traversing upward from the newly inserted node toward the root that violates the AVL property. Since an insertion involves adding a single node, this can only increase the height of some subtree by 1. Thus, the heights of the left and right subtrees differ by exactly 2. For any of the four cases (LL, RR, LR, RL), since the inserted node must be in the subtree of height $h+1$ of y, the height of the subtree whose root is x has height $h+2$ both before insertion and after rotation. Thus, the fact that the AVL property is satisfied for all ancestors of x before insertion implies that the AVL property is satisfied for all ancestors of x after rotation. □

The worst-case and amortized time complexities are $O(h) = O(\log n)$.

vi Deletion

Deletion in AVL tree involves first doing deletion the same as in BST, and then tracing the path from

- the original parent of the deleted node if the deleted node had zero or one child, and
- the original parent of the inorder successor or inorder predecessor that was moved if the deleted node had two children,

to the root: For each node x encountered, check if heights of its left and right subtrees differ by at most 1,

- if yes, proceed to the parent of x;
- if not, we say the AVL property is violated at x and perform rotations to restore it, and then proceed to the parent of x since violation may propagate upwards.

The worst-case and amortized time complexities are $O(h) = O(\log n)$.

vii Number of distinct AVL tree shapes

The number $A_{h,n}$ of distinct shapes of AVL trees with n nodes and height h is:

$$A_{-1,0} = 1$$

$$A_{-1,n} = 0, \quad n \neq 0$$

$$A_{0,1} = 1$$

$$A_{0,n} = 0, \quad n \neq 1$$

$$A_{h,n} = \sum_{i=0}^{n-1} A_{h-1,i}A_{h-1,n-1-i} + A_{h-1,i}A_{h-2,n-1-i} + A_{h-2,i}A_{h-1,n-1-i}, \quad h \in \mathbb{N}, n \in \mathbb{N}_0$$

Proof. The root uses one node, leaving $n-1$ nodes for the two subtrees. Since the whole tree has height h, the two trees must either have heights $h-1, h-2$ or both have height $h-1$. Thus,

$$A_{h,n} = \sum_{i=0}^{n-1} A_{h-1,i}A_{h-1,n-1-i} + A_{h-1,i}A_{h-2,n-1-i} + A_{h-2,i}A_{h-1,n-1-i}$$

□

viii Number of distinct AVL tree shapes with minimum number of nodes given height

The number B_h of distinct shapes of AVL trees with height h and minimum number of nodes is:

$$B_{-1} = 1$$

$$B_0 = 1$$

$$B_h = 2B_{h-1}B_{h-2}, \quad h \in \mathbb{N}$$

$$B_h = 2^{\frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^{h+2} - \left(\frac{1-\sqrt{5}}{2}\right)^{h+2}\right)-1}, \quad h+1 \in \mathbb{N}_0$$

Proof. To minimize the number of nodes, the two subtrees of root must be the AVL tree with height $h-1, h-2$ and minimum number of nodes, which have B_{h-1}, B_{h-2} distinct shapes respectively. There are two possible arrangements:

- Left subtree has height $h-1$, and right subtree has height $h-2$.
- Left subtree has height $h-2$, and right subtree has height $h-1$.

Thus,

$$B_h = 2B_{h-1}B_{h-2}$$

Let

$$C_h = \log_2 B_h$$

$$C_h = C_{h-1} + C_{h-2} + 1$$

Use characteristic equation:

$$x^2 - x - 1 = 0$$

Let

$$\varphi = \frac{1+\sqrt{5}}{2}$$

Note that

$$\frac{3+\sqrt{5}}{2} = \varphi^2 = \varphi + 1$$

$$2+\sqrt{5} = \varphi^3 = 2\varphi + 1$$

$$\frac{1-\sqrt{5}}{2} = -\frac{1}{\varphi} = 1 - \varphi$$

$$x = \frac{1 \pm \sqrt{5}}{2} = \varphi, -\frac{1}{\varphi}$$

$$C_h = a\varphi^h + b\left(-\frac{1}{\varphi}\right)^h + c$$

$$C_0 = a + b + c = \log_2 1 = 0$$

$$C_1 = a\varphi - \frac{b}{\varphi} + c = \log_2(2 * 1 * 1) = 1$$

$$C_2 = a\varphi^2 + \frac{b}{\varphi^2} + c = 0 + 1 + 1 = 2$$

$$C_2 = a(\varphi + 1) - b\left(\frac{1}{\varphi} - 1\right) + c = 2$$

$$C_2 - C_1 = a + b = 2 - 1 = 1$$

$$c = C_0 - (a + b) = 0 - 1 = -1$$

$$C_1 - c = a\varphi - \frac{1-a}{\varphi} = 1 + 1 = 2$$

$$a\left(\varphi^2 + 1\right) = 2\varphi + 1$$

$$a = \frac{2\varphi + 1}{\varphi + 2} = \frac{\varphi^3}{\varphi + 2}$$

$$b = 1 - a = \frac{1-\varphi}{\varphi+2} = -\frac{1}{\varphi(\varphi+2)}$$

$$C_h = \frac{1}{\varphi+2}\left(\varphi^{h+3} + \left(-\frac{1}{\varphi}\right)^{h+1}\right) - 1$$

$$\frac{1}{\varphi+2} = \frac{2}{5+\sqrt{5}} = \frac{1}{\sqrt{5}\varphi}$$

$$C_h = \frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^{h+2} - \left(\frac{1-\sqrt{5}}{2}\right)^{h+2}\right) - 1$$

$$B_h = 2^{\frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^{h+2} - \left(\frac{1-\sqrt{5}}{2}\right)^{h+2}\right)-1}$$

□

VII Splay tree

i Splay tree

A splay tree is a self-adjusting binary search tree (BST) that keeps no info of height and in which every access operation moves the accessed node to the root through an operation called splaying, invented by Daniel Dominic Sleator and Robert Endre Tarjan. The idea is locality, that is, if an item is accessed, it is likely to be accessed again soon. The worst-case searching, insertion, and deletion time complexities are as with a normal BST, while the amortized time complexities are $O(\log n)$.

ii (Bottom-up) splay

When a node x is accessed, a (bottom-up) splay operation consisting of a sequence of splay steps is performed on x to move it to the root, called splaying x. Let the parent of x be p and the parent of p be g. There are three types of splay steps, each of which has two symmetric variants:

- Zig step / simple rotation: This is done if x is a child of the root. We perform a rotation on the edge between x and root. This only happens as the last step and only happens when x originally has odd depth.
- Zig-zig step: This is done either when x is a left child of p and p is a left child of g, or when x is a right child of p and p is a right child of g. We perform a rotation on the edge between p and g and then a rotation on the edge between x and p.
- Zig-zag step: This is done either when x is a left child of p and p is a right child of g, or when x is a right child of p and p is a left child of g. We perform a rotation on the edge between x and p and then a rotation on the edge between x, now at the original position of p, and g.

A zig step is a 1-level splay step while a zig-zig or zig-zag step is a 2-level splay step.

iii Searching and finding minimum/maximum

Searching and finding minimum/maximum in splay tree involves first doing the operation as with a normal BST and then splay the found node if found.

iv Join

Given two BSTs S and T such that all elements of S are smaller than all elements of T, the following steps can be used to join them to a single BST: either

1. Splay the largest item in S. Now this item is in the root r of S and has no right child.
2. Set the right child of the new root r of S to the root of T.

or

1. Splay the smallest item in T. Now this item is in the root r of T and has no left child.
2. Set the left child of the new root r of T to the root of S.

v Split

Given a BST and a value x, splitting the BST at x returns two new BSTs, one containing all elements less than or equal to x, L, and the other containing all elements greater than x, R. This can be done with:

1. Search for x in the tree. If we found it, let y be that node; otherwise let y be the node we stop at.
2. Splay y.
3. Split its right subtree, which is R, from the rest of the tree, which is L.

vi Insertion

To insert a value x into a splay tree, there are two ways: either

1. Insert x as with a normal BST.
2. Splay x.

or

1. Split at x. However, if x is found in the tree, stop after splaying x.
2. Create a new tree with x as root and the two split trees as its subtrees.

vii Deletion

To delete a node x from a splay tree: either

1. Search for x.
2. Splay x.
3. Delete x and join the two subtrees of x.

or

1. Delete x as with a normal BST.
2. Splay the original parent of the deleted x.

viii Amortized analysis

We define size of a node $s(x)$ as the number of nodes in the subtree whose root is node x, rank of a node as

$$r(x) = \log_2 s(x)$$

and potential function of a tree $P(T)$ is

$$P(T) = \sum_{x \in T} r(x)$$

The actual cost C_i of an operation i is defined as the number of rotations in the operation. Let the state before operation i be denoted by the original functions and the state after operation i be denoted with a prime symbol. The amortized cost A_i of an operation i with actual cost C_i that transforms the tree T_{i-1} to T_i is defined as

$$A_i = C_i + \Delta P = C_i + P(T_i) - P(T_{i-1})$$

- Zig step:

$$C_i = 1$$

$$r'(p) \leq r(p)$$

$$\Delta P = r'(x) + r'(p) - r(x) - r(p) \leq r'(x) - r(x)$$

$$A_i \leq 1 + r'(x) - r(x)$$

- Zig-zig step:

$$C_i = 2$$

$$r'(p) \leq r(g)$$

$$r(x) \leq r(p)$$

$$s(x) + s'(g) = s'(x) - 2 \leq s'(x)$$

The log function is concave, i.e.,

$$\frac{\log_2 a + \log_2 b}{2} \leq \log_2\left(\frac{a+b}{2}\right)$$

i.e.,

$$log_2 a + \log_2 b \leq 2\log_2(a+b) - 2$$

Hence

$$r(x) + r'(g) = 2r'(x) - 2$$

$$r'(g) \leq 2r'(x) - r(x) - 2$$

$$\Delta P = r'(x) + r'(p) + r'(g) - r(x) - r(p) - r(g) \leq r'(x) + r'(g) - 2r(x) \leq 3(r'(x) - r(x)) - 2$$

$$A_i \leq 3(r'(x) - r(x))$$

- Zig-zag step:

$$C_i = 2$$

$$s'(p) + s'(g) = s'(x) - 1 \leq s'(x)$$

The log function is concave, i.e.,

$$\frac{\log_2 a + \log_2 b}{2} \leq \log_2\left(\frac{a+b}{2}\right)$$

i.e.,

$$log_2 a + \log_2 b \leq 2\log_2(a+b) - 2$$

Hence

$$r'(p) + r'(g) \leq 2r'(x) - 2$$

$$r(x) \leq r(p) \leq r(g)$$

$$\Delta P = r'(x) + r'(p) + r'(g) - r(x) - r(p) - r(g) \leq 3(r'(x) - r(x)) - 2$$

$$A_i \leq 3(r'(x) - r(x))$$

When splaying a node x, zero or more 2-level splay steps followed by zero or one 1-level splay step is conducted. Let $r_2(x)$ be the rank of x just after the last 2-level splay step and $r_f(x)$ be the final rank of x. The amortized cost A of splaying x is

$$\begin{aligned} A &\leq 1 + r_f(x) - r_2(x) + 3(r_2(x) - r(x)) \\ &\leq 1 + 3(r_f(x) - r_2(x)) + 3(r_2(x) - r(x)) \\ &= 1 + 3(r_f(x) - r(x)) \\ &\leq 1 + 3(\log_2 n - r(x)) \\ &= O(\log n) \end{aligned}$$

For a sequence of m accesses in a splay tree T, we can bound the total actual cost with

$$\sum_{i=1}^{m} C_i = P(T_0) - P(T_m) + \sum_{i=1}^{m} A_i$$

For every node x,

$$1 \leq s(x) \leq n$$

$$0 \leq r(x) \leq \log_2 n$$

Thus the change of rank $\Delta r(x)$ of a node x is

$$\Delta r(x) \leq \log_2 n$$

Thus the change of potential function is

$$P(T_0) - P(T_m) \leq \sum_{x \in T} \log_2 n = O\left(n \log n\right)$$

ix Weighted analysis

The above analysis can be generalized by assigning each node x a weight $w(x) \geq 1$ and defining $s(x)$ as the sum of weights of nodes in the subtree whose root is node x, called weighted analysis. Note that the previous analysis corresponds to the special case $w(x) = 1$. The sum of all weights is denoted W. The same analysis gives:

$$A \leq 1 + 3(\log_2 n - r(x)) = O(\log\left(\frac{W}{w(x)}\right) + 1)$$

For a sequence of m accesses in a splay tree T, we can bound the total actual cost with

$$\sum_{i=1}^{m} C_i = P(T_0) - P(T_m) + \sum_{i=1}^{m} A_i$$

For every node x,

$$w(x) \leq s(x) \leq W$$

$$\log_2(w(x)) \leq r(x) \leq \log_2 W$$

Thus the change of rank $\Delta r(x)$ of a node x is

$$\Delta r(x) \leq \log_2\left(\frac{W}{w(x)}\right)$$

Thus the change of potential function is

$$P(T_0) - P(T_m) \leq \sum_{x \in T} \log_2\left(\frac{W}{w(x)}\right) = O\left(\sum_{x \in T} \log\left(\frac{W}{w(x)}\right)\right)$$

x Balance theorem

The cost of performing a sequence of m accesses in a splay tree containing n nodes is

$$O(m \log n + n \log n)$$

Proof. Take a constant weight, e.g.,

$$w(x) = 1$$

Then

$$W = n$$

$$A_i = O(\log n)$$

$$P(T_0) - P(T_m) = O(n \log n)$$

□

This theorem implies that splay trees perform at least as well as AVL trees on sequences of at least n accesses.

xi Static optimality theorem

The cost of performing a sequence of m accesses in a splay tree T containing n nodes, with each node accessed at least once and q_x being the number of times node x is accessed, is

$$O\left(m + \sum_{x \in T} q_x \log\left(\frac{m}{q_x}\right)\right)$$

Proof. Let

$$w(x) = q_x$$

Then

$$W = m$$

$$A_i = O\left(\log\left(\frac{m}{q_x}\right) + 1\right)$$

$$\sum_{i=1}^{m} A_i = O\left(\sum_{x \in T} q_x \log\left(\frac{m}{q_x}\right) + m\right)$$

$$q_x \geq 1$$

$$P(T_0) - P(T_m) = O\left(\sum_{x \in T} \log\left(\frac{m}{q_x}\right)\right) = O\left(\sum_{x \in T} q_x \log\left(\frac{m}{q_x}\right)\right)$$

□

xii Working set theorem

The cost of performing a sequence S of m accesses in a splay tree T containing n nodes, with $t(x)$ being the number of distinct nodes accessed before the previous time node x was accessed for each $x \in S$, is

$$O\left(m + n \log n + \sum_{x \in S} \log(t(x) + 1)\right)$$

Proof. Let

$$w(x) = (t(x) + 1)^{-2}$$

Then

$$W = O(1)$$

$$A_i = O\left(\log(t(x) + 1) + 1\right)$$

$$\sum_{i=1}^{m} A_i = O\left(\sum_{x \in S} \log(t(x) + 1) + m\right)$$

$$P(T_0) - P(T_m) = O\left(n \log n\right)$$

□

xiii Locality property or strong balance theorem

The cost of performing a sequence of m accesses in a splay tree containing n nodes, with k distinct nodes accessed, is

$$O(m + m \log k + n \log n)$$

Proof. Use working set theorem.

$$t(x) = O(k)$$

$$O\left(m + n \log n + \sum_{x \in S} \log(t(x) + 1)\right) = O(m + n \log n + m \log k)$$

□

xiv Top-down splay

An alternative implementation of splay that is done together with searching. Suppose we want to splay x. We start at the root and maintain two trees which are initially empty, left tree L of nodes smaller than x and right tree R of nodes larger than x, during the traversal. In each step with current node c, there are three cases, each with two symmetric variants, where we will describe the former and the latter is the former with left and right exchanged and L and R exchanged:

- $x <$ left(c): We perform a rotation on the edge between c and left(c) and then a rotation one the edge left(left(c)) and left(c), and then assign left(c) to c.
- $x >$ right(c): We perform a rotation on the edge between c and right(c) and then a rotation one the edge right(right(c)) and right(c), and then assign right(c) to c.
- left$(c) < x < c$: We connect c into R, and then assign left(c) to c.
- right$(c) > x > c$: We connect c into L, and then assign right(c) to c.
- $x = c$, $x < c$ and c has no left child, or $x > c$ and c has no right child: The traversal stops at c. We take c as the new root, and L and R as left and right subtrees of c.

Top-down splay has the same worst-case and amortized searching, insertion, and deletion time complexities as bottom-up splay and $O(1)$ extra space.

VIII An implementation of binary tree, BST, ASL tree, and splay tree in C++

```
#pragma once
#include <algorithm>
#include <concepts>
#include <functional>
#include <stdexcept>
#include <utility>
#include <vector>
```

```cpp

template <class T, class Node> class BaseBinaryTreeNode {
public:
  T value;
  Node *parent;
  Node *left = nullptr;
  Node *right = nullptr;

  BaseBinaryTreeNode(const T &_value = T(), Node *_parent = nullptr)
      : value(_value), parent(_parent) {}

  virtual ~BaseBinaryTreeNode() = default;
};

template <class T>
class BinaryTreeNode : public BaseBinaryTreeNode<T, BinaryTreeNode<T>> {
  using Base = BaseBinaryTreeNode<T, BinaryTreeNode<T>>;
  using Base::Base;
};

template <class T, class Node = BinaryTreeNode<T>> class BinaryTreeStatic {
public:
  using node_type = Node;

  static std::vector<T> preorder(Node *node) {
    if (node == nullptr)
      return std::vector<T>();
    std::vector<T> result;
    std::vector<Node *> stack;
    stack.push_back(node);
    while (!stack.empty()) {
      Node *cur = stack.back();
      stack.pop_back();
      result.push_back(cur->value);
      if (cur->right != nullptr)
        stack.push_back(cur->right);
      if (cur->left != nullptr)
        stack.push_back(cur->left);
    }
    return result;
  }

  static std::vector<Node *> preorder_node(Node *node) {
    if (node == nullptr)
      return std::vector<Node *>();
    std::vector<Node *> result, stack;
    stack.push_back(node);
    while (!stack.empty()) {
      Node *cur = stack.back();
      stack.pop_back();
      result.push_back(cur);
      if (cur->right != nullptr)
        stack.push_back(cur->right);
      if (cur->left != nullptr)
        stack.push_back(cur->left);
    }
    return result;
  }

  static std::vector<T> inorder(Node *node) {
    if (node == nullptr)
      return std::vector<T>();
```

```
    std::vector<T> result;
    std::vector<Node *> stack;
    while (!stack.empty() || node != nullptr) {
      while (node != nullptr) {
        stack.push_back(node);
        node = node->left;
      }
      node = stack.back();
      stack.pop_back();
      result.push_back(node->value);
      node = node->right;
    }
    return result;
  }

  static std::vector<Node *> inorder_node(Node *node) {
    if (node == nullptr)
      return std::vector<Node *>();
    std::vector<Node *> result, stack;
    while (!stack.empty() || node != nullptr) {
      while (node != nullptr) {
        stack.push_back(node);
        node = node->left;
      }
      node = stack.back();
      stack.pop_back();
      result.push_back(node);
      node = node->right;
    }
    return result;
  }

  static std::vector<T> postorder(Node *node) {
    if (node == nullptr)
      return std::vector<T>();
    std::vector<T> result;
    std::vector<Node *> stack;
    Node *last_visited = nullptr;
    while (!stack.empty() || node != nullptr) {
      if (node != nullptr) {
        stack.push_back(node);
        node = node->left;
      } else {
        Node *right = stack.back()->right;
        if (right != nullptr && last_visited != right)
          node = right;
        else {
          result.push_back(stack.back()->value);
          last_visited = stack.back();
          stack.pop_back();
        }
      }
    }
    return result;
  }

  static std::vector<Node *> postorder_node(Node *node) {
    if (node == nullptr)
      return std::vector<Node *>();
    std::vector<Node *> result, stack;
    Node *last_visited = nullptr;
    while (!stack.empty() || node != nullptr) {
```

```
132       if (node != nullptr) {
133         stack.push_back(node);
134         node = node->left;
135       } else {
136         Node *right = stack.back()->right;
137         if (right != nullptr && last_visited != right)
138           node = right;
139         else {
140           result.push_back(stack.back());
141           last_visited = stack.back();
142           stack.pop_back();
143         }
144       }
145     }
146     return result;
147   }
148
149   // 1: left child
150   // 0: right child
151   // -1: node's parent is nullptr
152   // -2: node is nullptr
153   static signed char is_left_child(Node *node) noexcept {
154     if (node == nullptr)
155       return -2;
156     if (node->parent == nullptr)
157       return -1;
158     return node->parent->left == node;
159   }
160
161   static Node *leftmost_child(Node *node) {
162     if (node == nullptr)
163       return nullptr;
164     while (node->left != nullptr)
165       node = node->left;
166     return node;
167   }
168
169   static Node *rightmost_child(Node *node) {
170     if (node == nullptr)
171       return nullptr;
172     while (node->right != nullptr)
173       node = node->right;
174     return node;
175   }
176 };
177
178 template <class T, class Node = BinaryTreeNode<T>>
179 class BinaryTree : public BinaryTreeStatic<T, Node> {
180 public:
181   using node_type = Node;
182   using Base = BinaryTreeStatic<T, Node>;
183   using Base::inorder;
184   using Base::inorder_node;
185   using Base::is_left_child;
186   using Base::leftmost_child;
187   using Base::postorder;
188   using Base::postorder_node;
189   using Base::preorder;
190   using Base::preorder_node;
191   using Base::rightmost_child;
192
193   mutable Node *root; // Allow root->parent != nullptr
```

```
194
195   BinaryTree(Node *_root = nullptr) : root(_root) {}
196
197   BinaryTree(const BinaryTree &) = delete;
198
199   BinaryTree(BinaryTree &&other) noexcept
200       : root(std::exchange(other.root, nullptr)) {}
201
202   // O(n)
203   std::vector<T> preorder() const { return preorder(root); }
204   std::vector<Node *> preorder_node() const { return preorder_node(root); }
205   std::vector<T> inorder() const { return inorder(root); }
206   std::vector<Node *> inorder_node() const { return inorder_node(root); }
207   std::vector<T> postorder() const { return postorder(root); }
208   std::vector<Node *> postorder_node() const { return postorder_node(root); }
209
210   // O(h)
211   Node *leftmost_child() const { return leftmost_child(root); }
212   Node *rightmost_child() const { return rightmost_child(root); }
213
214 protected:
215   // O(1)
216   virtual Node *replace(Node *node, Node *replacement,
217                         bool deleting = false) const {
218     switch (is_left_child(node)) {
219     case -2:
220       throw std::invalid_argument("Node is nullptr");
221       break;
222     case 1:
223       node->parent->left = replacement;
224       break;
225     case 0:
226       node->parent->right = replacement;
227       break;
228     }
229     if (root == node)
230       root = replacement;
231     if (replacement != nullptr)
232       replacement->parent = node->parent;
233     Node *parent = node->parent;
234     if (deleting)
235       delete node;
236     return parent;
237   }
238
239   void recursive_clear(Node *node) noexcept {
240     if (node == nullptr)
241       return;
242     recursive_clear(node->left);
243     recursive_clear(node->right);
244     delete node;
245   }
246
247 public:
248   // recursive:
249   // >0: stack
250   // 0: try stack catch recursive
251   // <0: recursive
252   void clear(Node *node, signed char recursive = 0) noexcept {
253     if (node == nullptr)
254       return;
255     if (root == node)
```

```
256       root = nullptr;
257     switch (is_left_child(node)) {
258     case 1:
259       node->parent->left = nullptr;
260       break;
261     case 0:
262       node->parent->right = nullptr;
263       break;
264     }
265     auto stack_clear = [](Node *node) {
266       std::vector<Node *> stack = preorder_node(node);
267       for (auto it = stack.rbegin(); it != stack.rend(); ++it)
268         delete *it;
269     };
270     if (recursive > 0)
271       stack_clear(node);
272     else if (recursive == 0) {
273       try {
274         stack_clear(node);
275       } catch (...) {
276         recursive_clear(node);
277       }
278     } else
279       recursive_clear(node);
280   }
281
282   void clear() noexcept { clear(root); }
283
284   virtual ~BinaryTree() { clear(); }
285
286   BinaryTree &operator=(const BinaryTree &) = delete;
287
288   BinaryTree &operator=(BinaryTree &&other) noexcept {
289     if (this != &other) {
290       clear();
291       root = std::exchange(other.root, nullptr);
292     }
293     return *this;
294   }
295
296   // O(1)
297   virtual Node *rotate_right(Node *node) const {
298     if (node == nullptr || node->left == nullptr)
299       throw std::invalid_argument("Node is nullptr");
300     Node *child = node->left;
301     replace(node, child);
302     node->left = child->right;
303     if (child->right != nullptr)
304       child->right->parent = node;
305     child->right = node;
306     node->parent = child;
307     return child;
308   }
309
310   // O(1)
311   virtual Node *rotate_left(Node *node) const {
312     if (node == nullptr || node->right == nullptr)
313       throw std::invalid_argument("Node is nullptr");
314     Node *child = node->right;
315     replace(node, child);
316     node->right = child->left;
317     if (child->left != nullptr)
```

```
318       child->left->parent = node;
319     child->left = node;
320     node->parent = child;
321     return child;
322   }
323
324   // O(1)
325   virtual Node *rotate_parent(Node *child) const {
326     if (child == nullptr || child->parent == nullptr)
327       throw std::invalid_argument("Node is nullptr");
328     if (is_left_child(child) == 1)
329       return rotate_right(child->parent);
330     return rotate_left(child->parent);
331   }
332 };
333
334 template <class T, class Node = BinaryTreeNode<T>, class Compare = std::less<T>>
335   requires std::strict_weak_order<const Compare &, const T &, const T &>
336 class BST : public BinaryTree<T, Node> {
337 public:
338   using node_type = Node;
339   using Base = BinaryTree<T, Node>;
340   using Base::clear;
341   using Base::inorder;
342   using Base::inorder_node;
343   using Base::leftmost_child;
344   using Base::postorder;
345   using Base::postorder_node;
346   using Base::preorder;
347   using Base::preorder_node;
348   using Base::rightmost_child;
349   using Base::root;
350   using Base::rotate_left;
351   using Base::rotate_right;
352
353   [[no_unique_address]] Compare comp;
354
355 protected:
356   using Base::replace;
357
358 public:
359   BST() : BST(nullptr, Compare()) {}
360   explicit BST(const Compare &_comp) : BST(nullptr, _comp) {}
361   BST(Node *_root, const Compare &_comp) : Base(_root), comp(_comp) {}
362
363   BST(const BST &) = delete;
364   BST &operator=(const BST &) = delete;
365
366   BST(BST &&) = default;
367   BST &operator=(BST &&) = default;
368
369   // O(h)
370   virtual Node *find_min(Node *node) const { return leftmost_child(node); }
371   virtual Node *find_max(Node *node) const { return rightmost_child(node); }
372   virtual Node *find_min() const { return find_min(root); }
373   virtual Node *find_max() const { return find_max(root); }
374
375   // Return node, found
376   // O(h)
377   virtual std::pair<Node *, bool> search(const T &x) const {
378     if (root == nullptr)
379       return {nullptr, false};
```

```
    Node *node = root;
    while (true) {
      if (comp(x, node->value)) {
        if (node->left == nullptr)
          return {node, false};
        node = node->left;
      } else if (comp(node->value, x)) {
        if (node->right == nullptr)
          return {node, false};
        node = node->right;
      } else
        return {node, true};
    }
  }

  // O(h)
  Node *inorder_successor(Node *node) const {
    if (node == nullptr)
      throw std::invalid_argument("Node is nullptr");
    if (node->right != nullptr)
      return leftmost_child(node->right);
    if (node == root)
      return nullptr;
    Node *p = node->parent;
    while (p != nullptr && node == p->right) {
      if (p == root)
        return nullptr;
      node = p;
      p = p->parent;
    }
    return p;
  }

  // O(h)
  Node *inorder_predecessor(Node *node) const {
    if (node == nullptr)
      throw std::invalid_argument("Node is nullptr");
    if (node->left != nullptr)
      return rightmost_child(node->left);
    if (node == root)
      return nullptr;
    Node *p = node->parent;
    while (p != nullptr && node == p->left) {
      if (p == root)
        return nullptr;
      node = p;
      p = p->parent;
    }
    return p;
  }

  // O(h)
  virtual Node *ordered_after(const T &x) const {
    Node *result = nullptr;
    Node *node = root;
    while (node != nullptr) {
      if (comp(x, node->value)) {
        result = node;
        node = node->left;
      } else
        node = node->right;
    }
```

```
    return result;
  }

  // O(h)
  virtual Node *ordered_not_before(const T &x) const {
    Node *result = nullptr;
    Node *node = root;
    while (node != nullptr) {
      if (!comp(node->value, x)) {
        result = node;
        node = node->left;
      } else
        node = node->right;
    }
    return result;
  }

  // O(h)
  virtual Node *ordered_before(const T &x) const {
    Node *result = nullptr;
    Node *node = root;
    while (node != nullptr) {
      if (comp(node->value, x)) {
        result = node;
        node = node->right;
      } else
        node = node->left;
    }
    return result;
  }

  // O(h)
  virtual Node *ordered_not_after(const T &x) const {
    Node *result = nullptr;
    Node *node = root;
    while (node != nullptr) {
      if (!comp(x, node->value)) {
        result = node;
        node = node->right;
      } else
        node = node->left;
    }
    return result;
  }

  // Return node, inserted
  // O(h)
  virtual std::pair<Node *, bool> insert(const T &x) {
    if (root == nullptr) {
      root = new Node(x);
      return {root, true};
    }
    Node *node = root;
    while (true) {
      if (comp(x, node->value)) {
        if (node->left == nullptr) {
          node->left = new Node(x, node);
          return {node->left, true};
        }
        node = node->left;
      } else if (comp(node->value, x)) {
        if (node->right == nullptr) {
```

```
          node->right = new Node(x, node);
          return {node->right, true};
        }
        node = node->right;
      } else
        return {node, false};
    }
  }

  // Return rebalance_start
  // O(h)
  virtual Node *remove_node(Node *node) {
    if (node == nullptr)
      throw std::invalid_argument("Node is nullptr");
    if (node->left == nullptr) {
      return replace(node, node->right, true);
    }
    if (node->right == nullptr) {
      return replace(node, node->left, true);
    }
    Node *succ = inorder_successor(node);
    Node *rebalance_start;
    if (succ->parent != node) {
      rebalance_start = succ->parent;
      replace(succ, succ->right);
      succ->right = node->right;
      node->right->parent = succ;
    } else
      rebalance_start = succ;
    replace(node, succ);
    succ->left = node->left;
    succ->left->parent = succ;
    delete node;
    return rebalance_start;
  }

  // Return rebalance_start, removed
  // O(h)
  virtual std::pair<Node *, bool> remove(const T &x) {
    auto [node, found] = search(x);
    if (!found)
      return {node, false};
    return {remove_node(node), true};
  }
};

template <class T> class AVLNode : public BaseBinaryTreeNode<T, AVLNode<T>> {
public:
  using Base = BaseBinaryTreeNode<T, AVLNode<T>>;
  using Base::Base;
  using Base::left;
  using Base::right;

  mutable int height = 1; // leaf 1

  static int get_height(AVLNode *node) noexcept {
    return node == nullptr ? 0 : node->height;
  }

  static int bf(AVLNode *node) {
    return get_height(node->left) - get_height(node->right);
  }
```

```
566
567   void update_height() const {
568     height = 1 + std::max(get_height(left), get_height(right));
569   }
570 };
571
572 template <class T, class Node = AVLNode<T>, class Compare = std::less<T>>
573 class AVL : public BST<T, Node, Compare> {
574 public:
575   using node_type = Node;
576   using Base = BST<T, Node, Compare>;
577   using Base::Base;
578   using Base::root;
579
580   AVL() : Base() {};
581   AVL(const AVL &) = delete;
582   AVL &operator=(const AVL &) = delete;
583
584   AVL(AVL &&) = default;
585   AVL &operator=(AVL &&) = default;
586
587   Node *rotate_left(Node *node) const override {
588     Node *child = Base::rotate_left(node);
589     node->update_height();
590     child->update_height();
591     return child;
592   }
593
594   Node *rotate_right(Node *node) const override {
595     Node *child = Base::rotate_right(node);
596     node->update_height();
597     child->update_height();
598     return child;
599   }
600
601   // Rebalance until rebalanced once or parent == nullptr
602   // up to outer tree if root->parent != nullptr
603   virtual Node *rebalance(Node *node) const {
604     node->update_height();
605     int bf = Node::bf(node);
606     if (bf > 1) {
607       Node *child = node->left;
608       if (Node::bf(child) < 0) // LR
609         rotate_left(child);
610       return rotate_right(node); // LL, LR
611     }
612     if (bf < -1) {
613       Node *child = node->right;
614       if (Node::bf(child) > 0) // RL
615         rotate_right(child);
616       return rotate_left(node); // RL, RR
617     }
618     if (node->parent == nullptr)
619       return node;
620     return rebalance(node->parent);
621   }
622
623   std::pair<Node *, bool> insert(const T &x) override {
624     auto [node, inserted] = Base::insert(x);
625     if (inserted && node->parent != nullptr)
626       rebalance(node->parent);
627     return {node, inserted};
```

```
  }

  Node *remove_node(Node *node) override {
    Node *rebalance_start = Base::remove_node(node);
    Node *cur = rebalance_start;
    while (cur != nullptr) {
      cur = rebalance(cur)->parent;
    }
    return rebalance_start;
  }
};

template <class T, class Node = BinaryTreeNode<T>, class Compare = std::less<T>>
class Splay : public BST<T, Node, Compare> {
public:
  using node_type = Node;
  using Base = BST<T, Node, Compare>;
  using Base::Base;
  using Base::comp;
  using Base::is_left_child;
  using Base::root;
  using Base::rotate_parent;

protected:
  using Base::replace;

public:
  Splay() : Base() {};
  Splay(const Splay &) = delete;
  Splay &operator=(const Splay &) = delete;

  Splay(Splay &&) = default;
  Splay &operator=(Splay &&) = default;

  virtual Node *splay(Node *node) const {
    if (node == nullptr)
      return nullptr;
    while (node != root) {
      Node *parent = node->parent;
      Node *grandparent = parent->parent;
      // Zig
      if (parent == root)
        rotate_parent(node);
      // Zig-zig
      else if ((parent->left == node && grandparent->left == parent) ||
               (parent->right == node && grandparent->right == parent)) {
        rotate_parent(parent);
        rotate_parent(node);
      }
      // Zig-zag
      else {
        rotate_parent(node);
        rotate_parent(node);
      }
    }
    return node;
  }

  Node *find_min(Node *node) const override {
    return splay(Base::find_min(node));
  }

```

```
690   Node *find_max(Node *node) const override {
691     return splay(Base::find_max(node));
692   }
693
694   Node *find_min() const override { return find_min(root); }
695   Node *find_max() const override { return find_max(root); }
696
697   Node *ordered_after(const T &x) const override {
698     return splay(Base::ordered_after(x));
699   }
700
701   Node *ordered_not_before(const T &x) const override {
702     return splay(Base::ordered_not_before(x));
703   }
704
705   Node *ordered_not_after(const T &x) const override {
706     return splay(Base::ordered_not_after(x));
707   }
708
709   Node *ordered_before(const T &x) const override {
710     return splay(Base::ordered_before(x));
711   }
712
713   std::pair<Node *, bool> search(const T &x) const override {
714     auto [result, second] = Base::search(x);
715     return {splay(result), second};
716   }
717
718   std::pair<Node *, bool> insert(const T &x) override {
719     auto [result, second] = Base::insert(x);
720     return {splay(result), second};
721   }
722
723   virtual void join_right(Node *other) {
724     if (other == nullptr)
725       return;
726     if (root == nullptr) {
727       root = other;
728       other->parent = nullptr;
729       return;
730     }
731     find_max();
732     root->right = other;
733     other->parent = root;
734   }
735
736   virtual void join_left(Node *other) {
737     if (other == nullptr)
738       return;
739     if (root == nullptr) {
740       root = other;
741       other->parent = nullptr;
742       return;
743     }
744     find_min();
745     root->left = other;
746     other->parent = root;
747   }
748
749   // Not return rebalance_start but always return nullptr
750   Node *remove_node(Node *node) override {
751     if (node == nullptr)
```

```
752       throw std::invalid_argument("Node is nullptr");
753     Node *victim = splay(node);
754     Node *right = victim->right;
755     if (victim->left != nullptr) {
756       replace(victim, victim->left, true);
757       if (right != nullptr)
758         right->parent = nullptr;
759       join_right(right);
760     } else
761       replace(victim, right, true);
762     return nullptr;
763   }
764
765 protected:
766   virtual Splay split_right(Node *other_root) {
767     if (other_root == nullptr)
768       return Splay(nullptr, comp);
769     replace(other_root, other_root->left);
770     other_root->parent = nullptr;
771     other_root->left = nullptr;
772     return Splay(other_root, comp);
773   }
774
775   virtual Splay split_left(Node *other_root) {
776     if (other_root == nullptr)
777       return Splay(nullptr, comp);
778     replace(other_root, other_root->right);
779     other_root->parent = nullptr;
780     other_root->right = nullptr;
781     return Splay(other_root, comp);
782   }
783
784 public:
785   virtual Splay split_after(const T &x) {
786     return split_right(ordered_after(x));
787   }
788
789   virtual Splay split_not_before(const T &x) {
790     return split_right(ordered_not_before(x));
791   }
792
793   virtual Splay split_before(const T &x) {
794     return split_left(ordered_before(x));
795   }
796
797   virtual Splay split_not_after(const T &x) {
798     return split_left(ordered_not_after(x));
799   }
800 };
```